

Folding Cloth

Editorial

Task Author: Hanks Chong

1. Problem recap

We are given a strip of cloth made of N unit cells labelled $0, 1, \dots, N - 1$ from left to right. The cloth occupies a number of **columns**; each column is a vertical **stack** of one or more cells. Initially there are N columns, the i -th holding only cell i .

A **folding step** takes a contiguous block of x columns from the left end or the right end, rotates the whole block by 180° , and lays it **above** or **under** the remaining columns, aligned at the appropriate end. After placing, any vertical gap that appears inside a column is closed by pressing. There are four step types:

- **LA** x — take the leftmost x columns, place them **above** the rest;
- **LU** x — take the leftmost x columns, place them **under** the rest;
- **RA** x — take the rightmost x columns, place them **above** the rest;
- **RU** x — take the rightmost x columns, place them **under** the rest.

The 180° rotation is a genuine rotation: it reverses the left-to-right order of the columns in the block **and** the top-to-bottom order of the cells inside each of those columns.

We are given M steps as arrays T (the type, 0, 1, 2, 3 for **LA**/**LU**/**RA**/**RU**) and X (the length x). The steps are guaranteed to end with all N cells in a single column. We must output that column, top to bottom. Writing L_i for the number of columns after i steps, the limits are $1 \leq M < N \leq 500,000$ and $1 \leq X_i < L_i$.

The intended solution runs in $O(N)$ time. Below we build up to it through the subtasks.

2. Subtask 1 — direct simulation ($N \leq 2000$)

Simulate the process literally. Keep one stack of cell labels per column (a `vector<int>` per column, or a single array of stacks). Each step rotates the chosen block — reversing the column order and reversing each individual stack — and concatenates each rotated column onto the column it lands on.

A single step touches $O(N)$ cells and there are $O(N)$ steps, so the running time is $O(N^2)$, comfortable for $N \leq 2000$. This also makes a convenient reference brute force for stress-testing the faster solutions.

```
// One LA/LU step over the leftmost x columns (block lives in [L, L+x)).
for (int j = 0; j < x; j++) {
    // src is folded onto dst, the column it lands on after the rotation
    int src = L + j;
    int dst = L + 2 * x - 1 - j;
    // rotate the folded column 180 degrees, then stack it onto column dst:
    // above for an LA step, under for an LU step
    reverse(stk[src].begin(), stk[src].end());
```

```

    if (type == LA)
        stk[dst].insert(stk[dst].begin(), stk[src]...);
    else
        stk[dst].insert(stk[dst].end(), stk[src]...);
}
L += x;

```

The snippet above only covers the **non-overflowing** case $2x \leq L_i$, where the folded block lands entirely within the existing columns. A long fold, $2x > L_i$, overhangs the far end and spills into **new** columns there, so `dst = L + 2x - 1 - j` would run past the current rightmost column. Subtask 1 has no short-fold guarantee, so it does contain such folds. The simplest way to cover them — and what the reference brute force does — is to first apply the long-fold rewrite of Subtask 8. That rewrite is more than a local change of `x` and `t`: it also maintains a global `reversed` flag, reinterprets every later instruction accordingly, and reverses the final answer once at the end (see Subtask 8 for the details). With it in place, every fold satisfies $2x \leq L_i$ and the loop above applies unchanged.

3. Subtasks 2–5 — the case $X_i = 1$

When every fold has $x = 1$ the cloth keeps a very simple shape, and we can maintain the answer with deques instead of moving cells around.

Subtask 2 (LA only)

With only **LA 1** folds, the configuration is always: the leftmost column holds the prefix of cells $0, 1, \dots, a$ (as a set — their vertical order is scrambled by the repeated rotations, e.g. two folds give $[1, 0, 2]$, not $[0, 1, 2]$), and every other column holds a single cell. Each **LA 1** rotates that growing left column 180° and lays it on top of the next single cell (column $L + 1$), merging them into the new left column. Concretely, store the order of the stacked column in a deque; in those terms each fold:

- reverses the current stack (the 180° rotation), then
- appends the newly absorbed single cell.

Reversing then appending at the back is the same as **alternating** between pushing the new cell to the front and to the back of the deque, so we never pay for the reversal: keep a boolean “is the deque logically reversed” and push to the matching end. This is $O(1)$ per step, $O(N)$ overall.

Since the final order depends only on N , one may instead derive a closed-form pattern and print it directly.

Subtask 3 (LA/LU)

Same shape as above, but whether the new cell is glued to the **top** or the **bottom** of the stack now depends on the type: **LA** places the block above, **LU** below. With the “logically reversed” flag this is still a single front/back push per step.

Subtask 4 (LA/RA)

Now folds come from both ends, so the configuration is: a stacked column on the left holding a prefix of cells $0, \dots, a$, a stacked column on the right holding a suffix $b, \dots, N - 1$ (again each only as a set, in scrambled vertical order), and single cells in between. Maintain **two** deques, one per end, growing them inward. When the two ends become adjacent ($L + 1 = R$), the final fold joins them; concatenate the two deques (one of them reversed) in the order dictated by the last step.

Subtask 5 (all four types, $X_i = 1$)

Combine the two previous ideas: two dequeues, each with its own reversed flag, and the front/back choice determined by the fold type. Merge them when the two ends meet. Everything stays $O(N)$.

4. Subtasks 6–7 — short folds ($2X_i \leq L_i$)

For larger x the deque trick breaks down, but a cleaner structural idea takes over. We describe two equivalent $O(N)$ solutions.

The adjacency invariant

Lemma. If at any moment a column holds cells $A_0, A_1, \dots, A_{K-1}$ from top to bottom, then for every $0 \leq i \leq K - 2$ the cells A_i and A_{i+1} are **adjacent in the final ordering**.

This holds because folding never separates two cells that are currently stacked on top of each other: a rotation may flip a column upside down, and pressing only removes empty gaps, but vertically-touching cells stay touching forever. Hence the final single column is a path that visits every cell once, and the whole problem reduces to recovering that path.

Solution 1 — adjacency graph

Build an undirected graph on the N cells. We only ever need, for each currently active column, its **top** cell and its **bottom** cell — the interior of a stack is already pinned down by earlier edges.

When two columns are merged by a step, the rotation lines them up so that:

- for **LA** / **RA** (placed **above**) the two columns meet **top to top**, so we add an edge between the two top cells;
- for **LU** / **RU** (placed **under**) they meet **bottom to bottom**, so we add an edge between the two bottom cells.

After adding the edge we update the merged column's exposed top/bottom endpoints. Keeping the two exposed cells of each column in `pos[c]` makes every merge $O(1)$. In the reference code `pos[c].second` is the **top** cell and `pos[c].first` the **bottom** cell, so an **above** fold joins the two `.second` endpoints:

```
// LA: for each folded column j, find the column k it lands on, join tops.
for (int j = L; j < L + x; j++) {
    int k = 2 * L + 2 * x - j - 1;           // mirror position after rotation
    createEdge(pos[j].second, pos[k].second);
    pos[k].second = pos[j].first;             // new exposed endpoint of column k
}
L += x;
```

When all steps are done the graph is a simple path. Walk it from one endpoint of the final column (the cell with degree 1, i.e. the current **top** of the surviving column) to the other, emitting cells in order. That walk is the answer.

Solution 2 — tree of positions

A second view builds a rooted tree whose nodes are the **spatial column positions**. Whenever a column A is stacked onto a column B , make B the parent of A and label the child **type-1** if A went **above** and **type-2** if it went **under**. Children are kept in chronological order.

Let S_A be the cell order of A 's subtree, written bottom to top, once all folds touching the subtree are done. If the type-1 children in chronological order are $B_0, \dots, B_{K-1}$ and the type-2 children are $C_0, \dots, C_{P-1}$, then S_A is the concatenation

$$\text{reverse}(S_{C_{P-1}}), \dots, \text{reverse}(S_{C_0}), A, \text{reverse}(S_{B_0}), \dots, \text{reverse}(S_{B_{K-1}}).$$

A single DFS recovers the answer directly in top-to-bottom order. The nested reversals are handled by a parity bit. The root is visited at even parity; at a node whose depth has the **same** parity as the root, recurse into its **type-1** (above) children in reverse chronological order, emit the node, then recurse into its **type-2** (below) children in chronological order. At nodes of the **opposite** parity, swap the two groups: type-2 children in reverse chronological order, the node, then type-1 children in chronological order. (Reading the code below, the `else` branch is the same-parity-as-root case.)

```
void dfs(int u, int parity) {
    if (parity & 1) {
        for (int i = (int)child2[u].size() - 1; i >= 0; --i)
            dfs(child2[u][i], parity ^ 1);
        ans.push_back(u);
        for (int v : child1[u])
            dfs(v, parity ^ 1);
    } else {
        for (int i = (int)child1[u].size() - 1; i >= 0; --i)
            dfs(child1[u][i], parity ^ 1);
        ans.push_back(u);
        for (int v : child2[u])
            dfs(v, parity ^ 1);
    }
}
```

The tree can degenerate into a chain of depth $\Theta(N)$ — for instance when every fold uses $x = 1$ — so a naive recursive DFS overflows the default 8 MiB stack. Either enlarge the stack explicitly or convert the DFS to an explicit-stack loop.

Why this is $O(N)$

If $2X_i \leq L_i$ then the x folded columns each land on a **distinct** remaining column without sticking out past the far end, so $L_{i+1} = L_i - X_i$. Summing over all steps, from $L_0 = N$ down to $L_M = 1$,

$$X_0 + X_1 + \dots + X_{M-1} = N - 1.$$

The total number of column merges (edges, or tree links) is therefore $N - 1$, and both solutions run in $O(N)$ time and memory.

5. Subtask 8 — long folds (no extra constraints)

The only thing left is a step with $2X_i > L_i$: a fold that grabs **more than half** of the columns. Such a fold can wrap past the opposite end, so $L_{i+1} \neq L_i - X_i$ and the clean accounting above fails.

The fix is a symmetry observation:

Folding x columns over from one side (with $2x > L$) creates **exactly the same column merges** as the short fold of length $L - x$ from the **opposite** side applied to the **current**

cloth (note $2(L - x) < L$). The two differ only by a global mirror of the cloth (left↔right and top↔bottom) — which we do **not** perform now, but record in a flag and undo at the very end.

So we keep a global boolean **reversed** meaning “the cloth is currently stored mirrored on both axes”. For each incoming step:

1. if **reversed**, the user’s instruction must be reinterpreted in mirrored storage, which flips both the side bit and the above/under bit: **t ^= 3**;
2. if the (possibly reinterpreted) fold is long, $2x > L$, rewrite it to the equivalent short fold from the opposite side — **t ^= 2**, **x = L - x** — and toggle **reversed**.

After the rewrite every fold satisfies $2x \leq L$, so the subtask-7 machinery applies verbatim. Mirroring never changes **which** pairs of columns get merged, so the graph/tree is built correctly; we only need to undo the global mirror at the very end, reversing the produced ordering when **reversed** is set.

```
int t = T[i], x = X[i];
if (reversed) t ^= 3;           // reinterpret in mirrored storage
int len = R - L + 1;
if (x * 2 > len) {              // long fold -> equivalent short fold
    reversed ^= 1;
    t ^= 2;
    x = len - x;
}
// ... now 2x <= len: run the short-fold merge for type t ...
```

This completes a clean $O(N)$ solution for the full problem.

6. Reference implementation

The model solution combines the adjacency graph (Solution 1) with the long-fold rewrite. It runs in $O(N)$ time and uses $O(N)$ memory.

```
#include <algorithm>
#include <utility>
#include <vector>
using namespace std;

static const int MAXN = 500005;
static vector<int> adj[MAXN];

static inline void createEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

vector<int> fold(int N, vector<int> T, vector<int> X) {
    for (int i = 0; i < N; i++) adj[i].clear();

    vector<pair<int, int>> pos(N);    // pos[c] = (one endpoint, other endpoint)
    for (int i = 0; i < N; i++) pos[i] = {i, i};
```

```

int L = 0, R = N - 1, reversed = 0;
int M = (int)T.size();
for (int i = 0; i < M; i++) {
    int t = T[i], x = X[i];
    if (reversed) t ^= 3;
    int len = R - L + 1;
    if (x * 2 > len) { reversed ^= 1; t ^= 2; x = len - x; }

    if (t == 0) { // LA: join tops, fold from the left
        for (int j = L; j < L + x; j++) {
            int k = L * 2 + x * 2 - j - 1;
            createEdge(pos[j].second, pos[k].second);
            pos[k].second = pos[j].first;
        }
        L += x;
    } else if (t == 1) { // LU: join bottoms, fold from the left
        for (int j = L; j < L + x; j++) {
            int k = L * 2 + x * 2 - j - 1;
            createEdge(pos[j].first, pos[k].first);
            pos[k].first = pos[j].second;
        }
        L += x;
    } else if (t == 2) { // RA: join tops, fold from the right
        for (int j = R; j > R - x; j--) {
            int k = R * 2 - x * 2 - j + 1;
            createEdge(pos[j].second, pos[k].second);
            pos[k].second = pos[j].first;
        }
        R -= x;
    } else { // RU: join bottoms, fold from the right
        for (int j = R; j > R - x; j--) {
            int k = R * 2 - x * 2 - j + 1;
            createEdge(pos[j].first, pos[k].first);
            pos[k].first = pos[j].second;
        }
        R -= x;
    }
}

// The graph is a path; walk it from one endpoint of the final column.
vector<char> visited(N, 0);
vector<int> ans;
ans.reserve(N);
int cur = pos[L].second;
while (cur != pos[L].first) {
    ans.push_back(cur);
    visited[cur] = 1;
    cur = !visited[adj[cur][0]] ? adj[cur][0] : adj[cur][1];
}
ans.push_back(cur);
if (reversed) reverse(ans.begin(), ans.end());
return ans;
}

```

7. Summary of the progression

Subtasks	Complexity	Key idea
1 — small	$O(N^2)$	Simulate the folds directly.
2–5 — $X_i = 1$	$O(N)$	One or two deques with a “reversed” flag.
6–7 — $2X_i \leq L_i$	$O(N)$	Adjacency invariant; recover a path via a graph or a tree.
8 — full	$O(N)$	Rewrite long folds as mirrored short folds with a global flag.

Register Machine

Editorial

Task Author: Samuel Trajtenberg

1. Problem recap

We are given a *register machine* with 64 registers numbered 0 to 63, each holding a nonnegative integer and all initially 0. A program is a sequence of instructions, labelled 0, 1, 2, ... in input order, of three kinds:

- `INCREMENT x, i` — increase reg_x by 1, then process instruction i .
- `DECREMENT x, i, j` — if $\text{reg}_x = 0$ process instruction i ; otherwise decrease reg_x by 1 and process instruction j .
- `HALT` — stop.

Execution starts at instruction 0. When the machine halts it opens the chest iff $\text{reg}_0 = N$. The machine breaks if it runs more than 50 000 000 instructions or jumps to a non-existent instruction.

We must output a program (by calling `add_increment`, `add_decrement`, `add_halt`) that leaves exactly N in register 0, using *as few instructions as possible*, and at most 100. The interesting subtask is the last one: $1 \leq N \leq 1\,000\,000$ with the score increasing as the program gets shorter.

Throughout, we write reg_k for the value stored at register k .

2. Warm-up subtasks

Subtask 1 — $N \leq 99$

Since N is tiny, just emit N increments on register 0 followed by a halt. This is one instruction per unit, easily within budget for $N \leq 99$.

Subtask 2 — N a power of two

We need a way to multiply a register by 2. The following three-instruction loop does it:

```
0. DECREMENT 0, 3, 1
1. INCREMENT 1, 2
2. INCREMENT 1, 0
```

Each iteration removes 1 from register 0 and adds 2 to register 1, so when the loop exits register 1 holds twice the original register 0. Starting from a single increment and applying $\log_2 N$ doublings (alternating the source/target registers, or copying back) yields any power of two.

3. Subtask 3 — the full progression

This is where the problem becomes interesting. Below we walk through the sequence of ideas, each shaving instructions off the previous one, ending with the optimal construction. The instruction counts assume the worst case over $N \leq 10^6$ (i.e. up to 20 bits).

About 80 instructions (20 points)

Use the binary representation of N . Take $\log_2 N$ copies of the doubling loop, alternating between registers 1 and 2. Whenever the current power of two appears in the binary expansion of N , insert an extra

INCREMENT 0 into that loop so the corresponding power of two is also added to register 0. This costs at most 4 instructions per bit and builds N from lowest to highest bit.

Equivalently one can build from highest to lowest bit by placing the optional increments *between* the doubling loops rather than inside them:

```
INCREMENT Reg 0
Reg 1 <- 2 * Reg 0
INCREMENT Reg 1 (optional)
Reg 0 <- 2 * Reg 1
INCREMENT Reg 0 (optional)
Reg 1 <- 2 * Reg 0
...
```

Both variants use a similar number of instructions in the worst case.

About 70 instructions (30 points)

To produce two consecutive 1-bits it is cheaper to multiply by 4 and then *decrement once*, rather than incrementing twice. This needs slightly more care — the value before the $\times 4$ step must be one higher so the subtraction does not disturb higher-order bits — but it brings the cost down to 7 instructions per 2 bits.

About 60 instructions (40 points)

A different idea, resembling a binary counter:

```
0. INCREMENT 0, 1
1. DECREMENT 1, 2, 0
2. INCREMENT 1, 3
3. DECREMENT 2, 4, 0
4. INCREMENT 2, 5
5. DECREMENT 3, 6, 0
6. INCREMENT 3, 7
...
39. DECREMENT 20, 40, 0
40. HALT
```

It scans registers 1, 2, 3, ...: if a register holds 1 it resets it to 0 and jumps back to instruction 0; otherwise it sets it to 1 and moves on. If registers 1, ..., 20 are initialised to the binary representation of X , the counter runs exactly $X + 1$ times before halting. So we initialise the registers to the binary representation of $N - 1$ (at most 19 extra instructions) and let the counter run. This uses at most 60 instructions.

50 instructions (50 points)

The bottleneck is now the counter, at 2 instructions per bit. We shrink it by using a smaller counter and *reusing* it. The difficulty is that the machine has no call stack, so we must engineer our own “return”.

A first approach uses a register (say register 31) as a return address: the i -th “call” increments it i times, and a chain of decrements at the end of the routine returns to the correct caller. Splitting the binary representation of N into chunks of X bits, using an X -bit counter and multiplying register 0 by 2^X between chunks, gives 56 instructions at the optimum ($X = 5$ or $X = 7$).

We can return more cheaply. Consider:

```
0. INCREMENT 1, 1
1. DECREMENT 1, 3, 2
2. INCREMENT 2, i1
```

```

3. DECREMENT 2, 5, 4
4. INCREMENT 3, i2
5. DECREMENT 3, 7, 6
6. INCREMENT 4, i3
...

```

The first time the section starting at instruction 1 is entered it jumps to i_1 , the second time to i_2 , then i_3 , and so on. This returns in $2n - 1$ instructions instead of $O(n^2)$, and with $X = 2$ or $X = 3$ the whole solution fits in 50 instructions.

43 instructions (57 points)

With X so small, the counter is barely worth it: instead of laying out bits and summing them, we can just use `INCREMENT` instructions directly. Combining the return trick with “construct from highest to lowest bit” gives:

```

0. INCREMENT 2, N19
1. INCREMENT 0, 2

// Multiply by 2
2. DECREMENT 0, 5, 3
3. INCREMENT 1, 4
4. INCREMENT 1, 2
5. DECREMENT 1, 7, 6
6. INCREMENT 0, 5

// Jump back to either 1 or 2, depending on the bit of N
7. DECREMENT 2, 9, 8
8. INCREMENT 3, N18
9. DECREMENT 3, 11, 10
10. INCREMENT 4, N17
...
37. DECREMENT 17, 39, 38
38. INCREMENT 18, N3
39. DECREMENT 18, 41, 40
40. INCREMENT 19, N2
41. DECREMENT 19, 42, N1
42. INCREMENT 0, 43 (skip if N0 = 0)
43. HALT

```

Here each N_i is 1 if bit i of N is 1, and 2 otherwise. This uses only 43 instructions.

32 instructions (68 points)

We can perform the return even more efficiently. Use register 2 as a counter for the number of iterations so far; at the end of each loop increment it, then copy its value to register 3 with:

```

0. DECREMENT 2, 5, 1
1. INCREMENT 3, 2
2. INCREMENT 4, 0
3. DECREMENT 4, 5, 4
4. INCREMENT 2, 3

```

Now one decrement per branch suffices, reducing the cost from 2 to 1 instruction per loop (at a higher fixed cost). Combined with the previous idea this gives a 32-instruction solution.

25 instructions (75 points)

The final lever is base choice: we need not work in binary. With a larger base k we either pay extra increments for the last digit, or (if we reuse the increments at the start of the program) add a check so we do not shift on the last iteration. Weighing the number of iterations against the cost of multiplying register 0 by k :

Base k	No. iterations	Multiply by k	Total instructions
2	20	5	32
3	13	6	28
4	10	6	26
5	9	8	28
6	8	7	27
7	8	10	31
8	7	8	29
9	7	8	30
10	6	9	31

The best base is 4, at 26 instructions. With base 4 we can save one final instruction by handling the last digit specially:

- If it is 0, halt immediately.
- If it is 1, add one extra increment.
- If it is 2, increment the counter for the second doubling loop before the last iteration — this adds 2 to the final answer.
- If it is 3, add 1 to N initially (reducing to the 0 case) and append a single decrement at the end.

In every case the last digit costs just one instruction, bringing the total to 25.

4. The optimal solution — 23 instructions

Let us try to write the number digit by digit in an unspecified base b .

To do so, we will need a sequence of instruction that is able to multiply by b the number written in 0. The easiest way to do this is to first copy $8 * \text{reg}_0$ to reg_1 and then copy reg_1 to reg_0 , as follows:

```
x:      increment(1, x + 1)
x + 1:  increment(1, x + 2)
x + 2:  increment(1, x + 3)
...
x + b-1: increment(1, x + b)
x + b:   decrement(0, x + b + 1, x)
x + b+1: decrement(1, 0, x + b + 2)
x + b+2: increment(0, x + b + 1)
```

We notice that if we start execution from instruction $x + b$ with $\text{reg}_1 = 0$ we get exactly the behaviour described before: while possible we decrement from register 0 and perform b additions of register 1, then while possible we decrement from register 1 and perform a single addition on register 0. At the end the snippet jumps to instruction Q .

What's more, this snippet can actually already perform operations of type $\text{reg}_0 \leftarrow b \cdot \text{reg}_0 + d$ for all $0 \leq d < b$, if instead of starting execution from instruction $x + b$ we start from instruction $x + b - d$.

Therefore, $\lceil \log_b N \rceil$ accurate calls to this snippet will be enough to obtain our desired number.

We now just need a way to keep track of how many digits we have written.

Consider the following snippet:

```
0:      decrement(2, 3, 1)
1:      increment(3, 2)
2:      increment(4, 0)
3:      decrement(4, 5, 4)
4:      increment(2, 3)
5:      increment(2, 6)
6:      decrement(3, K0, 7)
7:      decrement(3, K1, 8)
...
6 + h:  decrement(3, Kh, 6 + h + 1)
7 + h:  halt()
```

When executed from instruction 0, with $\text{reg}_3 = \text{reg}_4 = 0$ the lines from 0 to 5 perform the assignment $(\text{reg}_2, \text{reg}_3) \leftarrow (\text{reg}_2 + 1, \text{reg}_2)$ (thus counting how many times we have executed the snippet) and then pass the execution to line 6. From line 6 onward we decrement register 3 as long as possible, and when it reaches 0 we can choose which instruction to jump to.

If we jump to appropriate positions of the first snippet, and then jump back to the second snippet by setting $Q = 0$, we can effectively write the digits of our number in register 0.

Once we have no more digits to write, we jump to a halt instruction.

For a naive implementation we would set $h = \lceil \log_b N \rceil - 1$, but we can actually save one last instruction by starting directly from the appropriate instruction inside the first snippet.

In this way we can have $h = \lceil \log_b N \rceil - 2$ and therefore use in total

$$6 + \lceil \log_b N \rceil + 3 + b = 9 + b + \lceil \log_b N \rceil$$

instructions.

A quick search reveals that for the optimal bases $b \in \{4, 5, 6\}$ this uses 23 instructions.

Out of these, base 4 has the easiest implementation. Below is a model solution in C++.

Model solution (base 4, 23 instructions) (Credit: Francesco Vercellesi)

```
#include <vector>
#include <tuple>
using namespace std;

void add_increment(int x, int i);
void add_decrement(int x, int i, int j);
void add_halt();

void program_machine(int N) {
    auto perm = [&](int i) {
        int start = 20 - ((N >> 18) & 3);
        if (i == 0) return start;
        if (i == start) return 0;
        return i;
    };
};
```

```

vector<tuple<int, int, int>> instr;
auto increment = [&](int x, int i) { instr.emplace_back(x, perm(i), -1); };
auto decrement = [&](int x, int i, int j) { instr.emplace_back(x, perm(i),
perm(j)); };
auto halt = [&] { instr.emplace_back(-1, -1, -1); };

decrement(2, 3, 1);
increment(3, 2);
increment(4, 0);
decrement(4, 5, 4);
increment(2, 3);
increment(2, 6);
decrement(3, 20 - ((N >> 16) & 3), 7);
decrement(3, 20 - ((N >> 14) & 3), 8);
decrement(3, 20 - ((N >> 12) & 3), 9);
decrement(3, 20 - ((N >> 10) & 3), 10);
decrement(3, 20 - ((N >> 8) & 3), 11);
decrement(3, 20 - ((N >> 6) & 3), 12);
decrement(3, 20 - ((N >> 4) & 3), 13);
decrement(3, 20 - ((N >> 2) & 3), 14);
decrement(3, 20 - ((N >> 0) & 3), 15);
halt();
increment(1, 17);
increment(1, 18);
increment(1, 19);
increment(1, 20);
decrement(0, 21, 16);
decrement(1, 0, 22);
increment(0, 21);

auto insert = [&](int idx) {
    auto [x, i, j] = instr[perm(idx)];
    if (x == -1) return add_halt();
    if (j == -1) return add_increment(x, i);
    add_decrement(x, i, j);
};

for (int i = 0; i < 23; i++)
    insert(i);
}

```

The `perm` helper swaps instruction 0 with the chosen entry point `start` of the multiply block, so that execution begins at the right digit position — this is the trick that lets us reuse the leading increments and reach $h = \lceil \log_b N \rceil - 2$.

5. Summary of the progression

Idea	Instructions	Key insight
Binary, increments in loops	80	4 instr / bit
Decrement after $\times 4$	70	7 instr / 2 bits
Binary counter	60	counter runs $X\{+\}1$ times
Reused counter + cheap return	50	return in $2n\{-\}1$

Idea	Instructions	Key insight
Direct increments, hi-to-lo	43	skip the counter
Copy-counter return	32	1 instr / loop
General base (base 4)	25	special last digit
Shared multiply + counter	23	reuse entry points